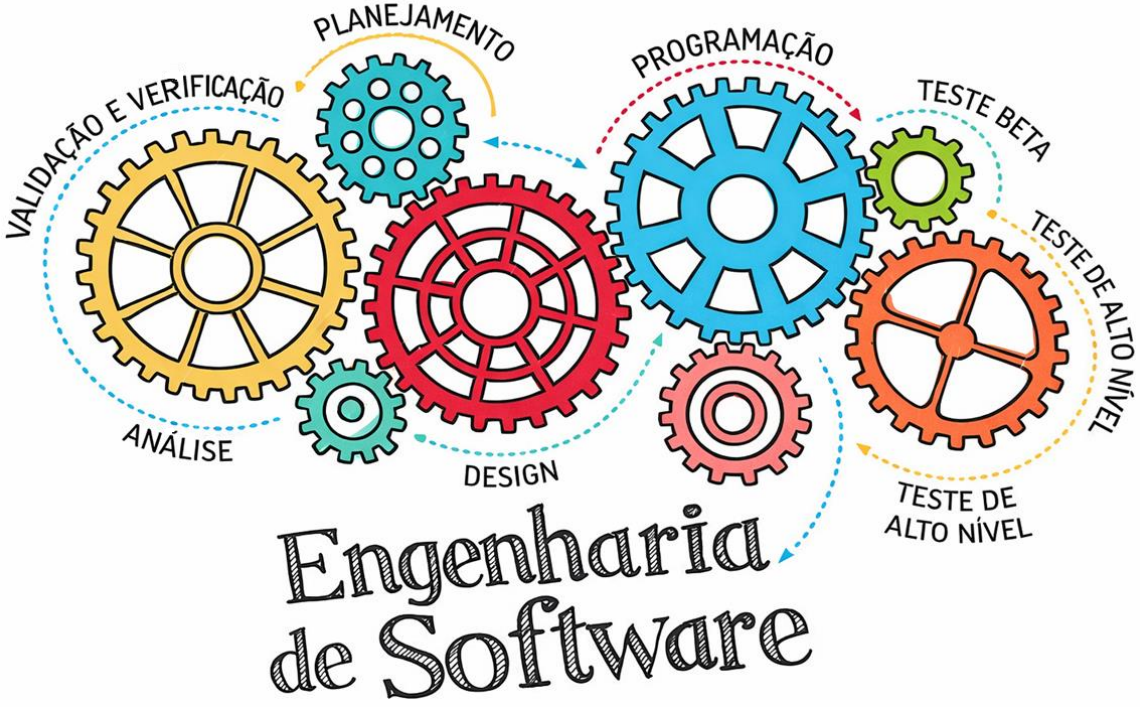
# MODELOS, MÉTODOS E TÉCNICAS DA ENGENHARIA DE SOFTWARE



ecossistema ănima

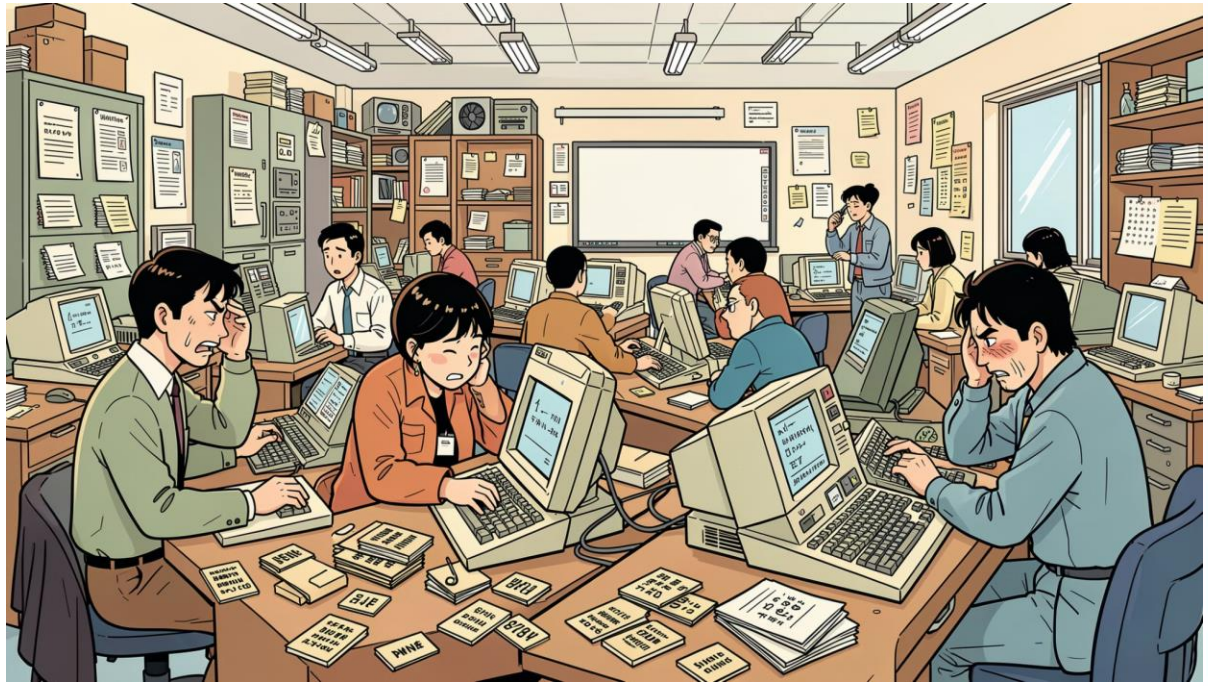
# Engenharia de Software – Conceitos e Práticas

- É uma disciplina da área de computação cujo o foco está em todos os aspectos da produção de software.
- Atuação desde os estágios iniciais da especificação do sistema até sua manutenção, quando o sistema já está sendo usado.
- O objetivo sempre é boa relação custo-benefício, aumento de qualidade, reduzindo prazos e custos.



# Por Que Surgiu a Engenharia de Software?

- Na década de 1960, o desenvolvimento de software era uma prática amadora e desorganizada, sem processos definidos ou metodologias estabelecidas.



**Projetos Atrasados**

Prazos constantemente ultrapassados com estouros significativos de orçamento

**Falhas Graves**

Software entregue frequentemente não atendia aos requisitos dos clientes

# A Crise do Software



Em 1968, a Conferência da OTAN na Alemanha cunhou o termo "Crise do Software", reconhecendo oficialmente que o desenvolvimento de software enfrentava desafios sistêmicos que afetavam organizações em todo o mundo.

**Prazos Não Cumpridos**

Projetos demoravam o dobro ou triplo do tempo estimado

**Requisitos Ignorados**

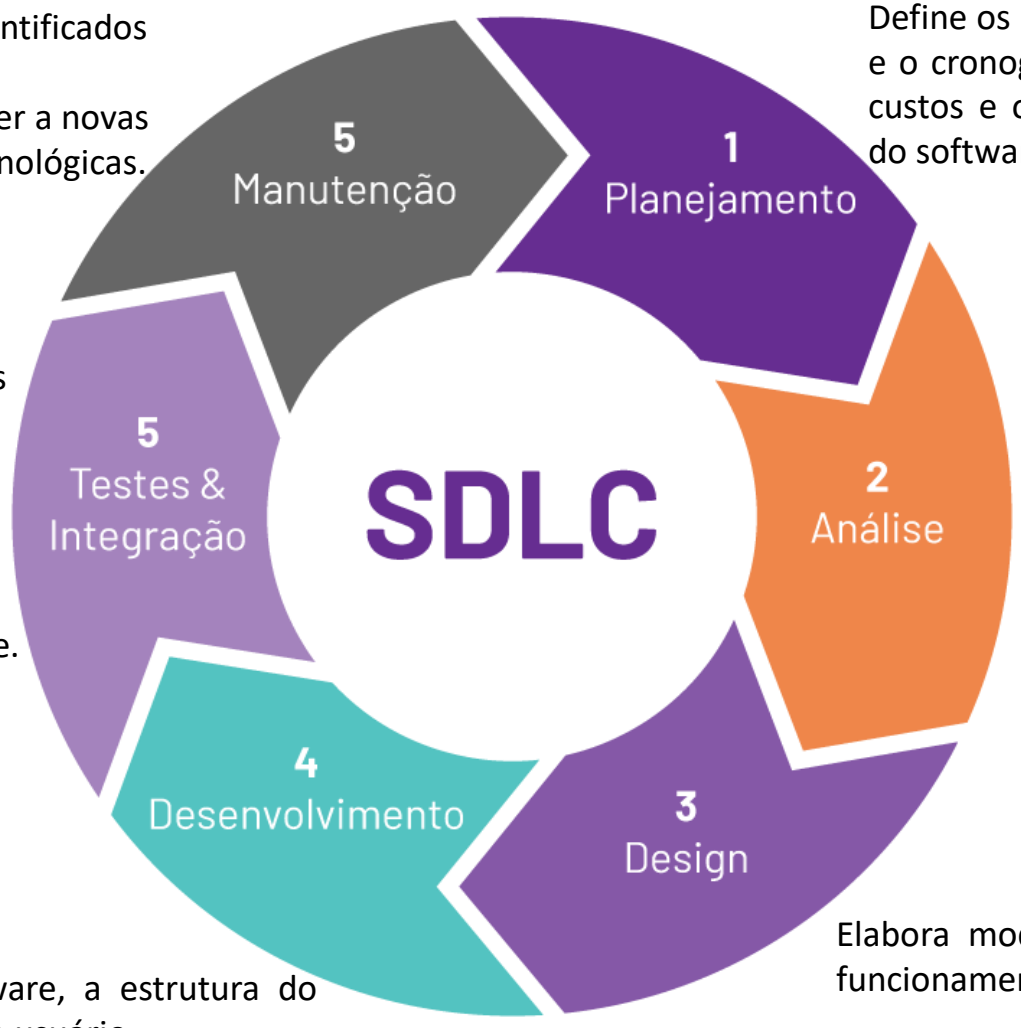
Funcionalidades essenciais eram negligenciadas ou mal implementadas

**Baixa Qualidade**

Sistemas instáveis causavam prejuízos para empresas e governos

# Ciclo de Vida do Desenvolvimento de Software

Define os objetivos, o escopo, os recursos necessários e o cronograma do projeto. Estabelece as etapas, os custos e os prazos para orientar o desenvolvimento do software.




Levanta e analisa as necessidades e expectativas dos usuários e clientes. Define os requisitos funcionais e não funcionais que o sistema deverá atender. Produz uma especificação dos requisitos para orientar as próximas etapas.

Elabora modelos e protótipos para representar o funcionamento e a aparência do sistema.

Define a arquitetura do software, a estrutura do banco de dados e a interface do usuário.
Transforma os projetos e as especificações em código-fonte utilizando linguagens de programação.

Verifica se as funcionalidades funcionam corretamente e atendem aos requisitos estabelecidos.
Integra os componentes e identifica erros, falhas e problemas de compatibilidade.

Realiza correções de erros identificados após a entrega do software.
Atualiza o sistema para atender a novas necessidades e mudanças tecnológicas.

# Requisito é

Uma condição ou uma capacidade necessária para um usuário (stakeholder) resolver um problema ou atingir um objetivo

Uma condição ou uma capacidade que deve ser provida por um sistema ou um componente do sistema para satisfazer um contrato, uma norma, uma especificação ou outro documento formal



# Fases de Requisitos

- Os requisitos devem ser levantados, esclarecidos, entendidos e revisados junto ao cliente.

**FASES:**

**Elicitação de requisitos**

**Análise e negociação dos requisitos**

**Definição e documentação dos requisitos**

**Validação dos requisitos**

# Tipos de Requisitos

## Requisitos Funcionais

**O QUE O SISTEMA DEVE FAZER**

- Um cliente deve poder fazer seu próprio cadastro.



- Um administrador deve poder cadastrar novos produtos ao sistema.






## Requisitos Não Funcionais

**COMO O SISTEMA DEVE FUNCIONAR**



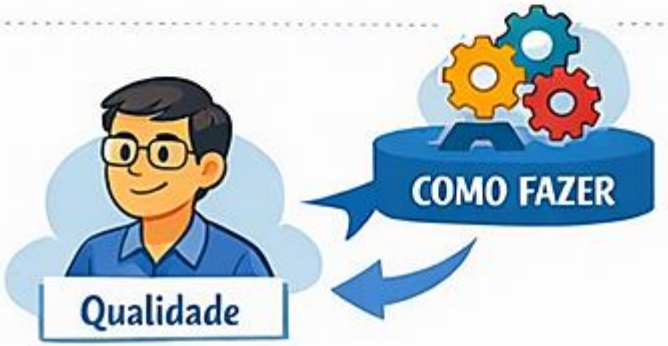
- O sistema deve processar um grande número de pesquisas sem lentidão.



- O sistema deve ter documentação completa e de fácil acesso.



- O sistema deve ter uma interface intuitiva e fácil de usar.



# Engenharia de Requisitos

## ENGENHARIA DE REQUISITOS

## A BASE DE TODO SOFTWARE

POR: LOJUAN BRYAN

### O QUE É?

**ENGENHARIA DE REQUISITOS**

É o processo de identificar, documentar e gerenciar as necessidades dos usuários para o desenvolvimento de um sistema.

### POR QUE?

**ELA É IMPORTANTE!**

Sem requisitos bem definidos, um projeto pode falhar por falta de entendimento do que realmente deve ser feito.




"60% das falhas em projetos vêm de requisitos mal definidos."

### TIPOS

**DE REQUISITOS**

**FUNCIONAIS**

DESCREVEM O QUE O SISTEMA FAZ.

EX: O SISTEMA DEVE PERMITIR O LOGIN DE USUÁRIOS.

**NÃO FUNCIONAIS**

DESCREVEM COMO O SISTEMA SE COMPORTA.

EX: O SISTEMA DEVE RESPONDER EM ATÉ 2 SEGUNDOS.

**FERRAMENTAS E TÉCNICAS**

- Entrevistas e reuniões
- Brainstorming
- Casos de uso (UML)
- Protótipos
- Reuniões de feedback

### ETAPAS DA ENGENHARIA DE REQUISITOS

1. **ELICITAÇÃO:** Coletar informações com o cliente.
2. **ANÁLISE:** Entender, priorizar e validar as necessidades.
3. **ESPECIFICAÇÃO:** Documentar de forma técnica e clara.
4. **VALIDAÇÃO:** Confirmar que o que foi definido reflete o que o cliente quer.
5. **GERENCIAMENTO:** Acompanhar mudanças e manter tudo atualizado.

### BENEFÍCIOS

- Clareza no escopo do projeto
- Menos erros e retrabalho
- Comunicação mais eficiente
- Software que realmente atende ao usuário

"UM REQUISITO BEM DEFINIDO É METADE DE UM SISTEMA BEM DESENVOLVIDO."
— IAN SOMMERVILLE



# Exemplo 1: Sistema de Biblioteca Online

Caso de Estudo: Criar um sistema de biblioteca online para uma universidade. O sistema deve permitir que os alunos pesquisem, reservem e renovem livros, além de permitir que os administradores gerenciem o acervo e os empréstimos.

**Requisitos Funcionais:**

- Permitir que os usuários pesquisem livros por título, autor ou categoria.
- Possibilitar o registro de novos usuários.
- Permitir que os usuários reservem livros disponíveis.
- Capacidade de os administradores adicionar, atualizar ou remover livros do acervo.
- Notificar os usuários sobre a disponibilidade de livros reservados.
- Permitir que os usuários renovem empréstimos de livros.
- O sistema deve garantir segurança e proteção dos dados dos usuários.

**Requisitos Não Funcionais:**

- Deve ter uma interface de usuário intuitiva e fácil de usar.
- Deve suportar múltiplos idiomas para atender a uma base diversificada de usuários.
- Deve ser capaz de lidar com um grande volume de usuários simultaneamente durante períodos de alta demanda, como início de semestre.
- O sistema deve estar disponível 24 horas por dia, 7 dias por semana, com um tempo de inatividade mínimo para manutenção programada.

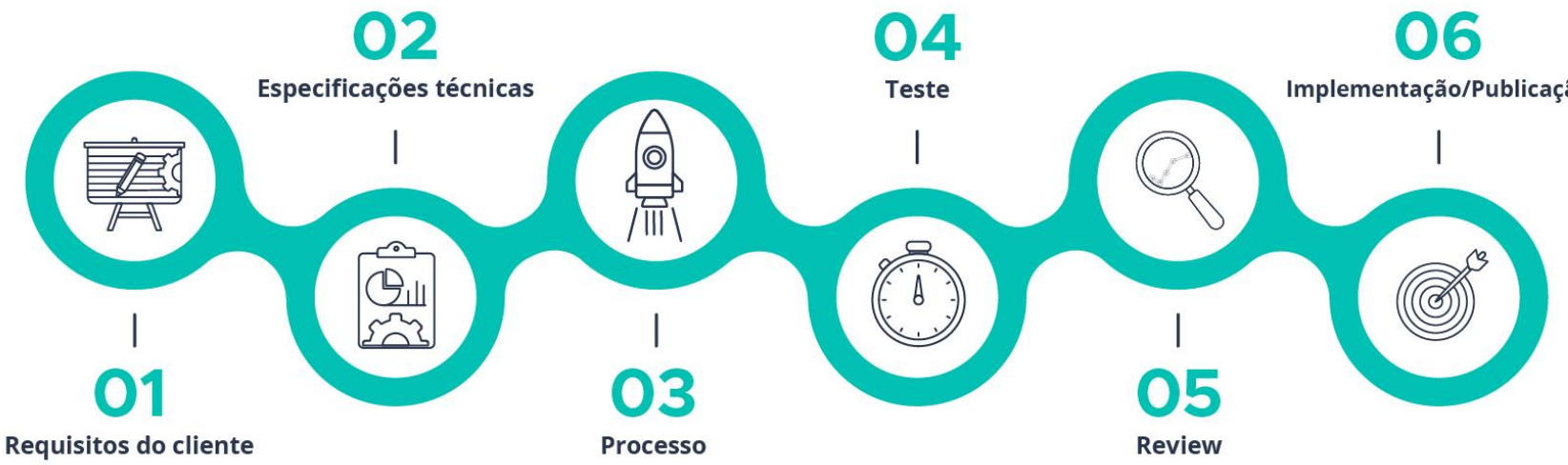
# Processo de Software

## Definição Fundamental

Um processo de software é um conjunto estruturado de atividades, métodos, práticas e transformações que as pessoas utilizam para desenvolver e manter um sistema de software e produtos associados.

## Pilares Essenciais

As atividades fundamentais incluem especificação (o que fazer), design e implementação (como fazer), validação (verificação da correção) e evolução (adaptação ao longo do tempo).



# Modelos de Processos de Software

- Um modelo é uma simplificação e representação da realidade.
- São construídos para permitir um melhor entendimento sobre o sistema que está sendo construído.
- Especifica a estrutura e comportamento, além de servir de guia para  a construção do sistema.
- Documentam as decisões tomadas.



# Modelo x Processo

- Um modelo é algo teórico, um conjunto de possíveis ações.
- O processo deve determinar ações práticas a serem realizadas pela equipe com prazos e métricas definidos.

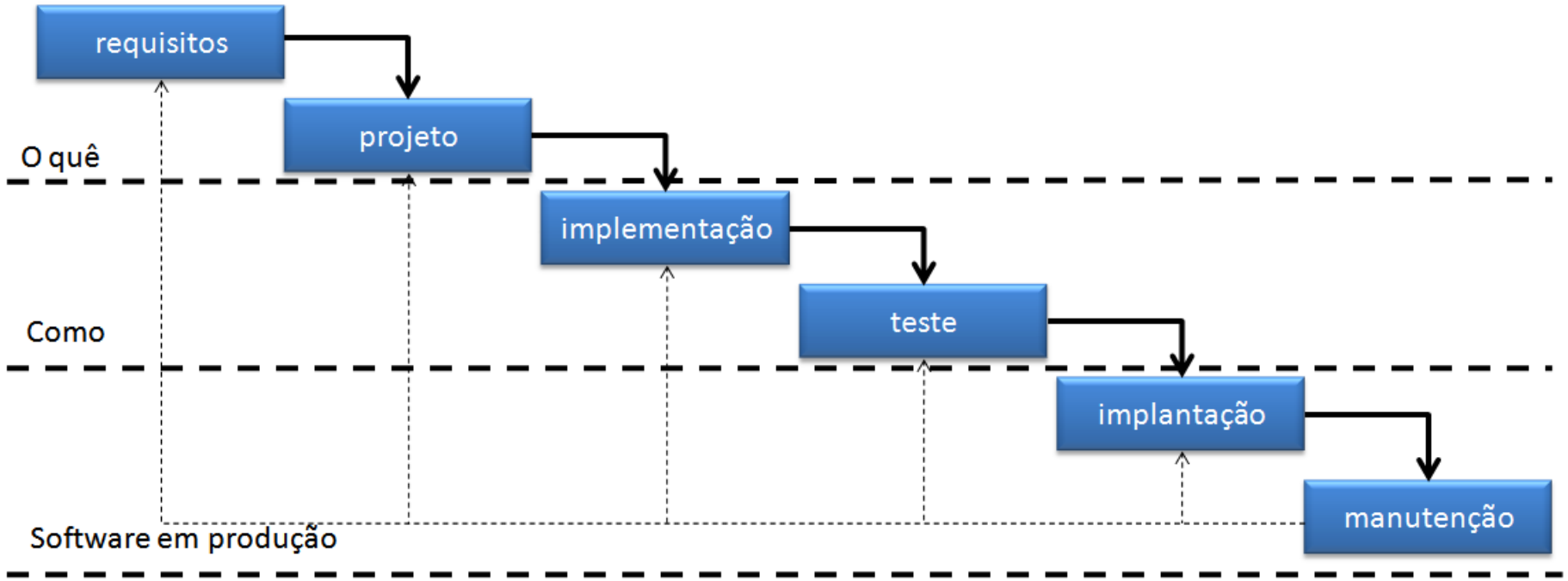
**Modelo + Planejamento = Processo**

# Modelos de Processos de Software

- **Modelo** = representação simplificada do processo
- 1. Modelo Cascata
- 2. Prototipação
- 3. Modelo Incremental
- 4. Modelo Espiral

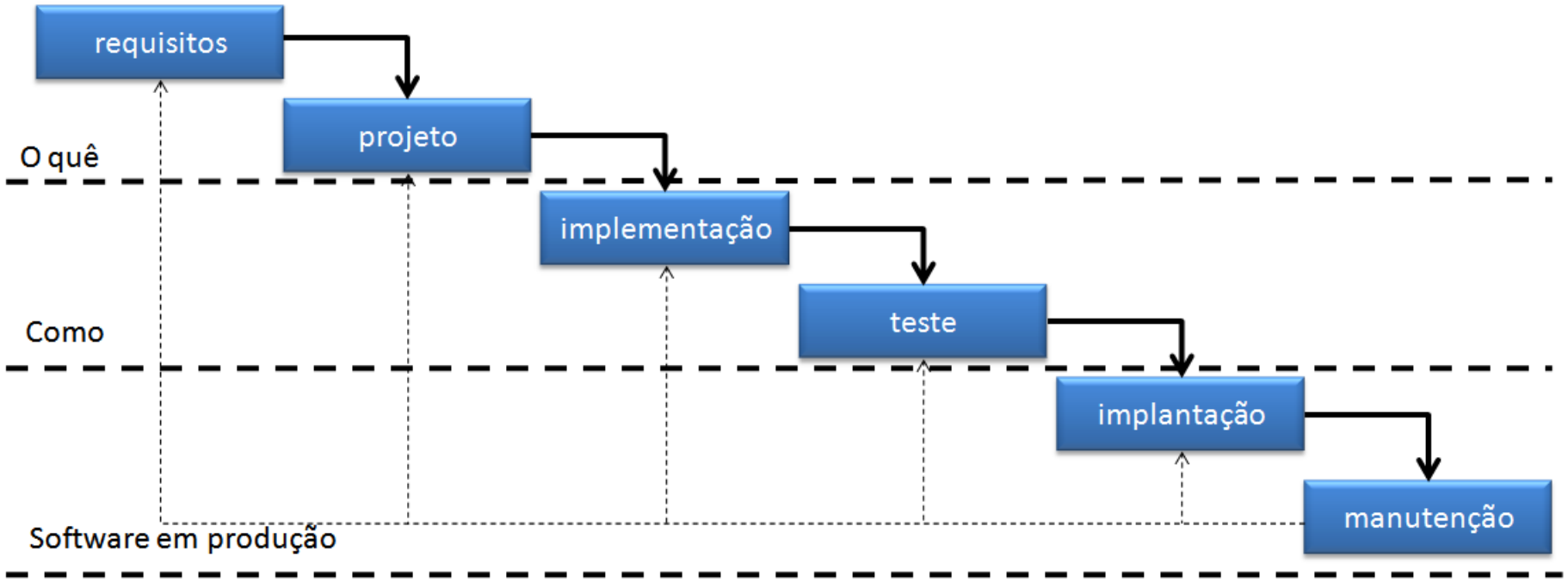
**Modelos Tradicionais**

# Modelo Cascata




- Primeiro modelo publicado do processo de desenvolvimento de software.
- Retrata um desenvolvimento gradual e possui uma sequência de passos em ordem que devem ser seguidos.

# Modelo Cascata




**Vantagens:**

- Documentação produzida em cada fase.
- O fluxo das atividades é extremamente organizado e muito mais fácil de ser gerenciado.

**Desvantagens:**

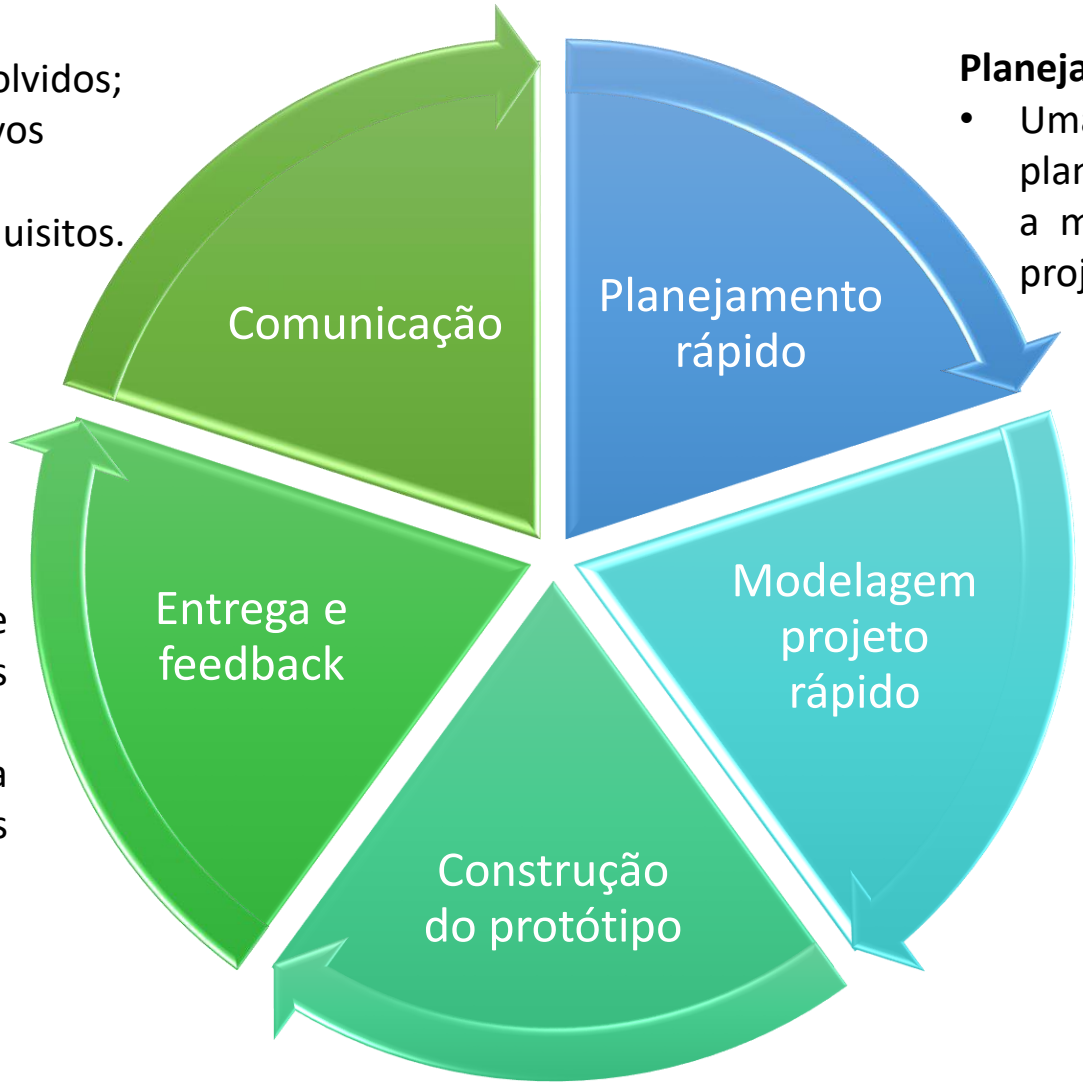
- Projetos raramente seguem fluxo sequencial.
- Dificuldade do cliente estabelecer todos os
- requisitos inicialmente.
- Pouca flexibilidade para mudanças.

# Prototipação – Modelo de Processo Evolucionário

- Produz uma versão cada vez mais completa do software a cada iteração;
- Software evolui ao longo do tempo;
- Requisitos do negócio e do produto mudam;
- Modelo iterativo (progride através de refinamentos).

# Prototipação

**Comunicação:**
- Reunião com os envolvidos;
- Definição dos objetivos gerais do software;
- Identificação dos requisitos.




**Planejamento rápido:**
- Uma iteração de prototipação é planejada rapidamente e ocorre a modelagem na forma de um projeto rápido.

**Modelagem projeto rápido:**
- Concentra-se em uma representação dos aspectos do software que serão visíveis para os usuários.

**Construção de protótipo:**
- Construção do protótipo propriamente dito.

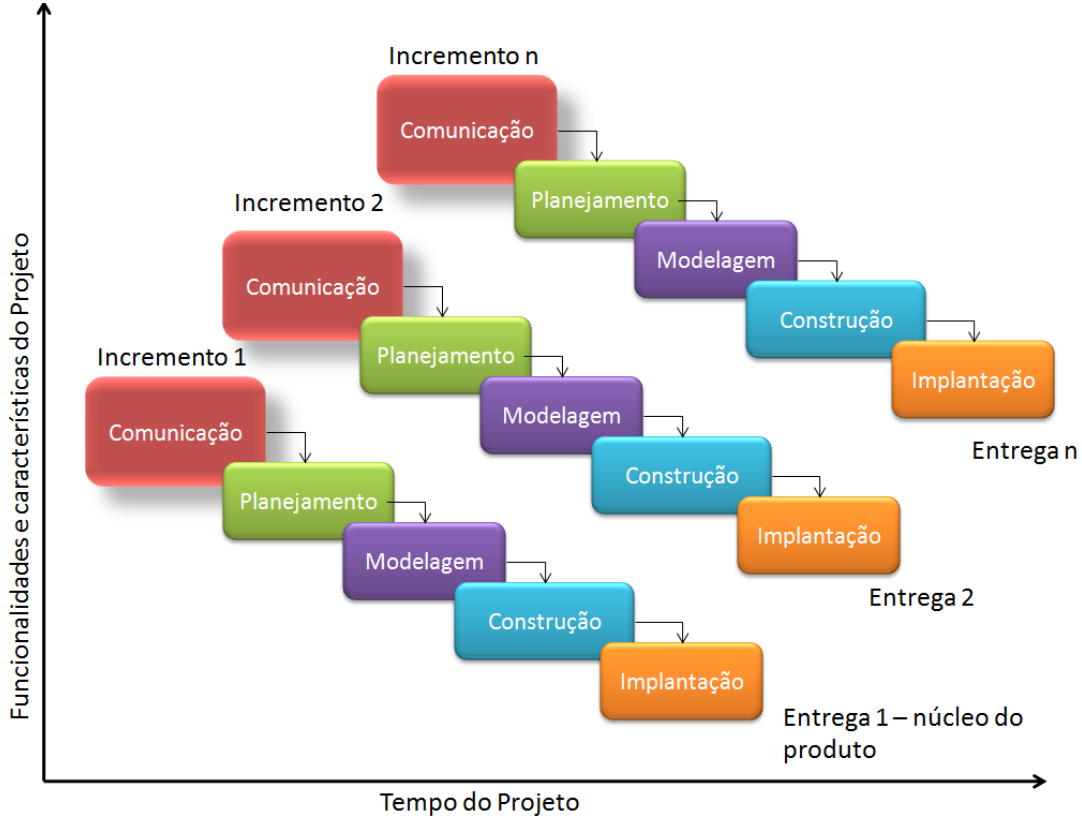
**Entrega e feedback:**
- O protótipo é entregue para todos os envolvidos e avaliados por eles;
- O feedback é usado para refinar ainda mais os requisitos.

# Prototipação

| VANTAGENS | DESVANTAGENS |
|---|---|
| Este modelo é flexível em design. | Este modelo é caro. |
| É fácil detectar erros. | Possui documentação deficiente devido às constantes mudanças nos requisitos do cliente. |
| Podemos encontrar facilmente a funcionalidade ausente. | Pode haver muita variação nos requisitos. |
| Existe um escopo de refinamento, o que significa que novos requisitos podem ser facilmente acomodados. | Os clientes às vezes exigem que o produto real seja entregue logo após ver um protótipo inicial. |
| Ele pode ser reutilizado pelo desenvolvedor para projetos mais complicados no futuro. | Pode haver soluções abaixo do ideal devido à pressa dos desenvolvedores em construir protótipos. |
| Garante um maior nível de satisfação e conforto do cliente. | Os clientes podem não estar satisfeitos ou interessados no produto depois de ver o protótipo inicial. |
| Ajuda os desenvolvedores e usuários a entender melhor o sistema. | Não há certeza na determinação do número de iterações. |
| Os requisitos de integração são muito bem compreendidos e os canais de implantação são decididos muito no início. | Pode haver uma análise de problema incompleta ou inadequada, que pode aumentar a complexidade do sistema. |
| Ele pode envolver ativamente os usuários na fase de desenvolvimento. | |

# Modelo Incremental

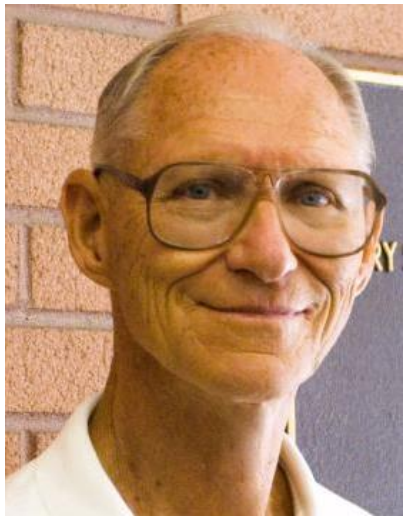
- Surge como uma combinação do modelo cascata com o de prototipação.
- Ao invés de especificar e desenvolver tudo de uma só vez, este modelo trabalha com incrementos, ou seja, pequenos pedaços de software entregues de cada vez.
- Quando uma parte do software fica pronta ela é incrementada às partes prontas ou quase prontas.



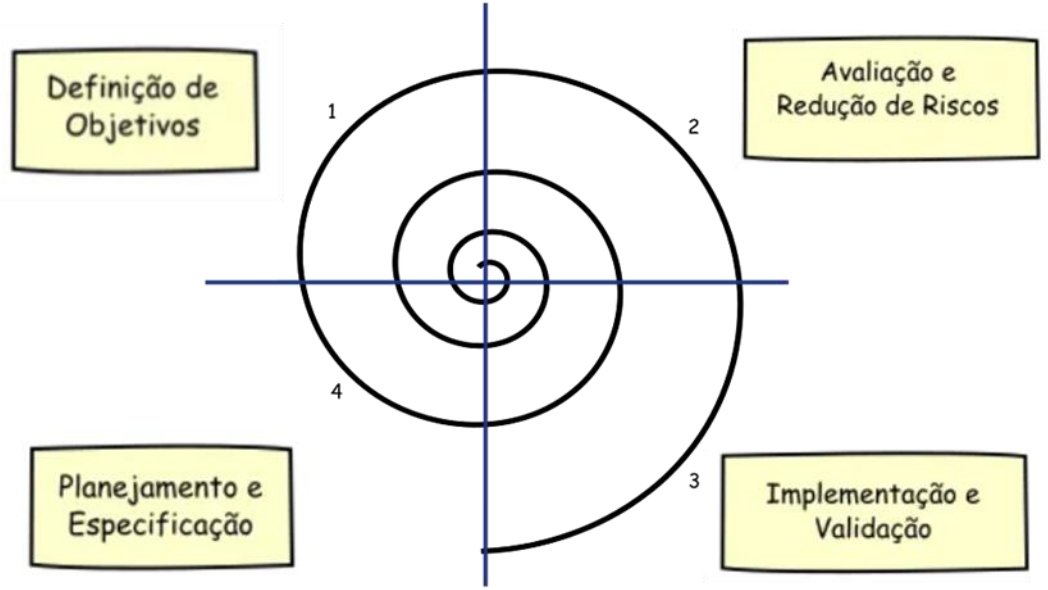
# Modelo Incremental

| VANTAGENS | DESVANTAGENS |
| --- | --- |
| **Entrega rápida de funcionalidades**: O software pode ser entregue em partes menores, com funcionalidades básicas funcionando desde o início. | **Complexidade da arquitetura**: A integração dos módulos incrementais pode ser complexa, levando a uma arquitetura desorganizada. |
| **Redução de riscos**: Permite identificar e corrigir problemas mais cedo, evitando grandes falhas no projeto. | **Dificuldade em estimar o tempo total**: É difícil estimar o tempo total do projeto, pois cada módulo pode ter um tempo de desenvolvimento diferente. |
| Flexibilidade: Facilita a adaptação a mudanças nos requisitos do cliente. | **Custos de integração**: Pode haver custos adicionais para integrar os módulos incrementais. |
| **Gerenciamento eficiente**: Permite dividir o projeto em partes menores e gerenciá-lo de forma mais eficiente. | **Falta de visão geral**: O cliente pode ter dificuldade em ter uma visão geral do software final, pois ele é entregue em partes. |
| **Melhor controle de qualidade**: Permite testar cada módulo incrementalmente, garantindo a qualidade do software final. | **Risco de retrabalho**: Mudanças nos requisitos podem levar a retrabalho em módulos já desenvolvidos. |

# Modelo Espiral de Boehm

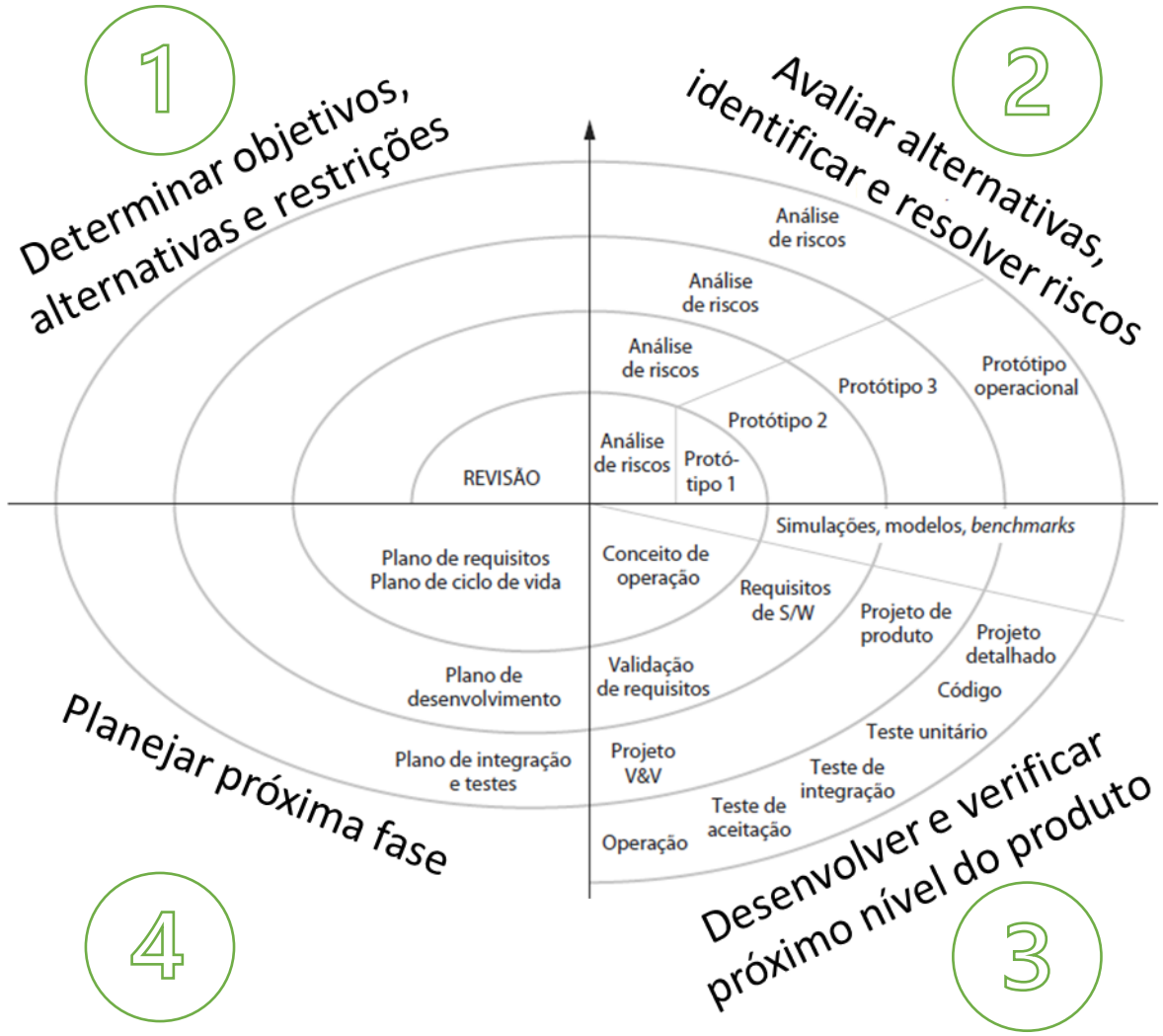


- Criado por Barry Boehm em 1988, professor de Engenharia de Software e diretor fundador do Centro de Engenharia de Sistemas e Software da Universidade do Sul da Califórnia.
- Modelo orientado a riscos que combina a natureza iterativa da prototipagem com os aspectos controlados e sistemáticos do modelo em cascata.



# Modelo Espiral

- Abordagem realista do desenvolvimento de sistemas e softwares de grande porte usando a prototipagem como mecanismo de redução de riscos.
- É um processo evolucionário, adequado para softwares que precisam passar por inúmeras evoluções na medida que o desenvolvimento acontece.
- O ciclo seguinte, com definição de requisitos; o seguinte, com o projeto do sistema, e assim por diante.



# Modelo Espiral

| VANTAGENS | DESVANTAGENS |
|---|---|
| Abordagem bem realista. | Pode ser difícil convencer os clientes que o processo de evolução é controlável, pois ele exige competência considerável na avaliação dos riscos e depende dessa competência para ser bem sucedido; |
| Como o modelo exige a consideração dos riscos técnicos em todos os estágios de evolução, se for aplicado adequadamente, reduzirá os riscos antes que se tornem problemáticos; | Se um risco importante não for descoberto e gerenciado corretamente, fatalmente ocorrerão problemas; |
| As estimativas se tornam mais realistas e o tempo de implementação é reduzido; | A avaliação dos riscos exige um analista com experiência; |
| Mais versátil para testar e lidar com mudanças; | Não deve ser usado em sistemas de pequeno porte. Aplica-se melhor a sistemas de grande porte; |
| Não faz distinção entre desenvolvimento e manutenção. | Erros na avaliação de riscos podem impactar o projeto. |

# BIBLIOGRAFIA BÁSICA


- RANGEL, Pablo. Sistemas orientados a objetos. Rio de Janeiro: Brasport, 2021. E-book. Disponível em: https://plataforma.bvirtual.com.br/Leitor/Publicacao/197367/epub/0.
- PRESSMAN, Roger; MAXIM, Bruce. Engenharia de Software. Uma abordagem profissional. 8a. Ed. Bookman, 2016. Disponível em: https://integrada.minhabiblioteca.com.br/reader/books/9786558040118/epubcfi/6/2[%3Bvnd.vst.idref%3DCapa.xhtml]!/4/2[page_i]/2%4051:1
- ELMASRI, Ramez. Sistemas de banco de dados. 6. ed. São Paulo: Pearson, 2011. Disponível em: http://unibh.bv3.digitalpages.com.br/users/publications/9788579360855